



SECURITY

DOC NO. RL-370-SECURITY-A REV. A RIPOSTE LABORATORIES

Read this before you treat glitchlock as encryption.

CONTENTS

▸ What it actually is	2
▸ What it is not	3
▸ If you need actual confidentiality	4
▸ Reasonable uses	4
▸ Public-key recipients	5
▸ Key handling	6
▸ Reporting	6

SEC.01

WHAT IT ACTUALLY IS

A keyed, reversible transposition-and-substitution cipher over selected fields of a video bitstream. The key is genuinely required: the transform plan is derived from HMAC-SHA256 over (key, nonce, feature, stream, frame), integers are drawn without modulo bias, and there is no shortcut back without the key. A wrong key produces garbage, not a partial recovery.

The manifest is authenticated with HMAC-SHA256 under the same key, so tampering with a layer parameter or a repair entry is detected before unlocking starts.

SEC.02

WHAT IT IS NOT

It is not confidentiality for the video's content. This is the important part and no amount of good key handling changes it.

The ciphertext is, by design, a valid playable video. That is the whole appeal, and it is also the whole problem:

- ▶ **The residual image survives.** Locking `mv` and `qscale` scrambles motion and quantisation. It does not touch the DCT coefficients, which carry the actual picture. Intra-coded blocks still decode to roughly the right colours in roughly the right places. Look at the sample image in the README: it is thoroughly wrecked, and you can still tell it is colour bars.
- ▶ **Structure leaks.** Frame count, frame types, resolution, GOP structure, bitrate profile and scene-cut positions are all untouched and readable.
- ▶ **The plaintext distribution is known.** Real motion vector fields are smooth and heavily concentrated near zero. An attacker knows this. A permutation within a frame preserves the multiset of values exactly, and additive substitution preserves nothing but is applied over a domain as small as 32 values. Neither is designed to resist statistical cryptanalysis, and neither does.
- ▶ **Known-plaintext is devastating.** If an attacker has the carrier and the locked file, the offsets and permutation fall out directly for that file. Reusing a `(key, nonce)` pair across files then compromises the others – same failure mode as reusing a stream cipher keystream. `glitchlock` generates a fresh random nonce per lock, so do not go out of your way to defeat that.
- ▶ **It is not authenticated encryption.** The manifest MAC covers the **manifest**. It does not cover the locked video's contents. Someone can edit the locked file and you will find out at unlock time via the carrier digest check – which is integrity detection after the fact, not AEAD.

SEC.03

IF YOU NEED ACTUAL CONFIDENTIALITY

Encrypt the file with a real tool – `age`, `gpg`, or anything AEAD-based – and use glitchlock for what it is good at. If you want both, encrypt the locked file; the two compose fine.

SEC.04

REASONABLE USES

- ▶ Reversible glitch art: destroy it on purpose, get it back on demand.
- ▶ Obfuscating footage for review or transport where the point is "not casually watchable" rather than "cryptographically secret".
- ▶ Screener or watermarking experiments where a keyed, reversible perturbation is the interesting property.
- ▶ Research on codec bitstream structure – the domain model and the round-trip harness are reusable on their own.

SEC.05

PUBLIC-KEY RECIPIENTS

-recipient adds hybrid encryption over the top: a random content key drives the scrambler, and that key is sealed to each recipient with X25519 → HKDF-SHA256 → ChaCha20-Poly1305, one fresh ephemeral keypair per recipient per lock. The manifest carries only the sealed copies.

What it fixes: **key distribution**. You no longer need a shared secret, and the manifest can travel with the video without carrying anything that opens it.

What it does not fix: everything in the section above. The ciphertext is still a playable video, the residual picture still survives, and the structure still leaks. Wrapping the content key in X25519 does not change one bit of what the locked video shows. If a reader concludes "it uses public keys, so it must be safe to publish the locked file", that conclusion is wrong.

Two more limits worth stating plainly:

- ▶ **No sender authentication.** This is a sealed box. Anyone holding your public key can lock a file **to** you, and the manifest does not prove who did. If you need to know who sent it, you need a signature layer, which does not exist yet.
- ▶ **The fingerprint authorises nothing.** It is a lookup hint so unsealing can try the right entry first. Access is decided solely by whether the X25519 exchange and the AEAD tag work out. Forging a fingerprint gains an attacker nothing, and there is a test for exactly that.

SEC.06

KEY HANDLING

- ▶ `-key-file` hashes the file's contents to 32 bytes; any file works, including a binary blob from `/dev/urandom`.
- ▶ `-password` derives a key with scrypt ($N=2^{15}$, $r=8$, $p=1$) using a per-lock random salt stored in the manifest.
- ▶ `GLITCHLOCK_KEY` is read from the environment if no other key is given.
- ▶ `-keyless` stores the seed in the manifest in the clear. This is not a weaker key, it is **no key**: anyone holding the manifest can unwind the file. It exists because reversible glitch art often does not want key management at all. It is labelled in the manifest as `"keyless": true` and printed by the CLI.

Lose the key and the file is gone. There is no recovery path and that is intentional.

SEC.07

REPORTING

Open an issue. If you find a case where `unlock` does not reproduce the carrier byte-for-byte, that is a correctness bug and the most serious kind here – please include the codec, the FFglitch version, and the manifest.

SOURCE SECURITY.tex SHA 0189835a051d BUILT 2026-09-07